

Atelier 05 : TD3 CSS : HTML/CSS Avancé (2/2)

Atelier 05 : TD2 CSS : HTML/CSS Avancé (2/2)

Développement Web : HTML/CSS

BTS 1^{er} année

TD3 : HTML/CSS Avancé 2/2

display et mise-en-page

1. Ordre d'application des sélecteurs CSS

Comme vous vous en souvenez, les sélecteurs servent à sélectionner un ensemble de balises sur lesquels on applique une règle CSS. Nous avons appris lors du TD1 les sélecteurs de base et lors du TD2 la combinaison de sélecteurs.

Plusieurs règles CSS peuvent porter sur un même élément HTML. Si ces règles peuvent coexister, elles sont toutes appliquées. Par exemple, si vous avez le code CSS suivant :

```
div { background-color:blue; }  
.skill { color:red; }
```

alors un `<div class="skill">` aura les deux propriétés `background-color:blue` et `color:red`.

Mais il peut aussi arriver que ces dernières soient contradictoires. Par exemple, si vous avez le code CSS suivant, **de quelle couleur** sera le texte d'un `<div>` de classe `skill` ?

```
.skill { color:red; }  
div { color:blue; }
```

Pour régler ces conflits, le CSS définit une notion de priorité basée sur l'emplacement des règles CSS puis sur la spécificité des sélecteurs CSS. Dans l'exemple précédent, c'est la première règle qui prévaut comme nous le verrons dans la suite.

1.1 Différents emplacements

Il y a plusieurs emplacements possibles pour déclarer du style CSS :

1. **Style externe** : (Conseillé) On peut utiliser un fichier de style externe et le lier au document HTML avec la balise `<link>` dans le `<head>` :

```
<html>  
<head>  
  <link rel="stylesheet" type="text/css" href="css/styles.css">  
</head>  
<body> ... </body>  
</html>
```

2. **Style interne** : (Déconseillé) On peut inclure des règles CSS directement dans le `<head>` à l'aide de la balise `<style>` :

- c compte le nombre de sélecteurs de classes (e.g. `.skill`) ou pseudo-classes (e.g. `:hover`, `:visited`, ...)
- d compte le nombre de sélecteurs de balises (e.g. `div`, `span`) ou de pseudo-éléments (e.g. `::first-letter`, `::after`)
- les opérateurs de combinaison et le sélecteur universel `*` ne contribuent pas à la priorité.

Pour revenir à l'exemple précédent, les règles ont donc comme priorité (en supposant qu'elles sont écrites dans un fichier de style externe) :

```
div /* -> (1,0,0,1) (un sélecteur de balise div) */
.skill /* -> (1,0,1,0) (un sélecteur de classe skill) */
```

1.3L'ordre de priorité

Il reste maintenant à expliquer comment on classe les priorités (a,b,c,d) . L'ordre sur les priorités est défini comme l'ordre du dictionnaire (ordre *lexicographique*) :

- on regarde d'abord la "première lettre" a . Si a est strictement plus grand alors la règle CSS est plus prioritaire. En cas d'égalité,
- on regarde la "deuxième lettre" b pour déterminer la priorité. Si b est strictement plus grand (et donc a égal) alors la règle CSS est plus prioritaire. En cas d'égalité sur a et b ,
- on regarde la "troisième lettre" c . En cas d'égalité,
- on regarde la "quatrième lettre" d .
- **en cas d'égalité absolue**, la règle écrite en dernier est prioritaire.

Ce mécanisme de priorité s'appelle la cascade et correspond au C de CSS (Cascading Style Sheet).

Exemple : `div.skill` (priorité $(1,0,1,1)$) est plus prioritaire que `div` (priorité $(1,0,0,1)$) car on a égalité sur a et b mais c est plus grand pour `div.skill`.

Exercice 1

Dans un fichier texte ou sur papier, écrivez les priorités des sélecteurs suivants et classez-les du plus prioritaire au moins prioritaire. On suppose que toutes ces règles sont définies dans un fichier externe, donc $a=1$, et les valeurs recherchées commencent toujours par $(1,...)$.

```
.titi span
div span
nav.titi .tata div div div div div
ul li div.skill
#id
div > a
div + a
```


Exercice 2

Quelle est la couleur du texte “Priorité CSS” dans les deux cas suivant ? Quelle règle de priorité CSS explique votre réponse ?

1. Le fichier **styles.css** contient **p {color:blue;}**, et le fichier **index.html** contient

```
<html>
<head>
  <style type="text/css">
    p {color: pink}
  </style>
  <link rel="stylesheet" type="text/css" href="css/styles.css">
</head>
<body><p style="color:red">Priorité CSS</p></body>
</html>
```

2. Le même fichier **styles.css** et le fichier **index.html** sans la règle inline

```
<html>
<head>
  <style type="text/css">
    p {color: pink}
  </style>
  <link rel="stylesheet" type="text/css" href="css/styles.css">
</head>
<body><p>Priorité CSS</p></body>
</html>
```

2.La propriété display

Comme nous l’avons vu au TD précédent, à chaque balise correspond quatre boîtes (*content*, *padding*, *border* et *margin*, voir la section sur le modèle de boîte du TD précédent).



La façon dont ces boîtes vont occuper l'espace est gérée par l'attribut **display**. Nous allons voir dans cette partie les trois valeurs principales de la propriété **display**.

2.1 display: block

Les éléments block sont des éléments :

- qui par défaut occupent toute la largeur,
- qui provoquent un saut de ligne avant et après son affichage (que l'on ait diminué sa largeur ou pas)
- dont on peut définir la taille en CSS via les propriétés **height** et **width**.

En pratique, on utilise des éléments de display **block** :

- dès que l'on veut expliciter l'agencement (layout) de certains éléments HTML d'une page. Par exemple nous voulons que l'en-tête (header) ait une hauteur de **200px**, qu'un paragraphe prenne **50%** de la largeur,...
- dès qu'il est naturel de prendre toute la place par défaut (exemple un titre **<h2>**).

Notez que les balises de structure que l'on a présentées au TD précédent ont **display: block** comme style par défaut dans le navigateur, ce qui explique qu'elles s'empilent verticalement comme on l'avait expliqué.

2.2 display: inline

Les éléments inline sont des éléments :

- qui prennent leur taille en fonction de leur contenu,
- qui ne provoquent pas un saut de ligne, ils sont écrits comme du texte les uns à la suite des autres
- dont on ne peut pas définir la taille en css via les propriétés **height** et **width**,

En pratique, on utilise des éléments de display **inline** :

1. dans du texte, pour ajouter de la sémantique sans interrompre la lecture du lecteur (mettre en exposant un nombre par **<sup>**, préciser l'importance d'une partie du texte par ****, associer un lien avec **<a>**),
2. lorsque l'on veut positionner des éléments à la suite.

Puisqu'associés au texte (****, **<a>**, ...), on trouve en majorité les éléments **inline** comme feuilles de l'arborescence du HTML.

Notez que les balises au niveau du texte que l'on a présentées au TD précédent ont **display: inline** comme style par défaut dans le navigateur, ce qui explique qu'elles se comportent comme du texte.

2.3 Exemple

Le code HTML suivant

Exercice 3

Dans ce premier exercice, nous allons créer un menu dans un style **block**.

1. Changez le menu de votre site par le suivant

```
<nav>
  <div><a href="/index.html">Accueil</a></div>
  <div><a href="/facts.html">Facts</a></div>
  <div><a href="/news.html">Actualités</a></div>
  <div><a href="/contact.html">Contact</a></div>
</nav>
```

2. Puisque **<nav>** est **display:block** par défaut (le vérifier sur Chrome si possible), il doit prendre toute la largeur. **Inspectez** donc votre **<nav>** pour voir si c'est le cas. Que constatez-vous ?

Explication : En fait, un élément **display:block** ne prend pas toute la largeur de **<body>** mais de son plus proche **block** parent. Ici, le père de **<nav>** est le bloc **<header>** et donc **<nav>** prend toute la largeur de **<header>**. Le plus proche **block** parent s'appelle le *containing block*.

3. Puisque **<nav>** est de type **block**, nous pouvons fixer ses dimensions. Donnez-lui la largeur **75%**. Que constatez-vous ?

Explication : La largeur d'une balise en **display:block** est relative à celle de son *containing block* (**<header>** ici). **Inspectez** **<header>** puis **<nav>** pour connaître leur largeurs. **Vérifiez** qu'on a bien un rapport de **75%**.

4. Pour centrer le **<nav>** dans son parent **<header>**, on va lui donner des marges horizontales **auto** (gardez les marges verticales à 0).

Rappel : Nous avons vu dans le dernier TD comment centrer horizontalement. Pour centrer un **display:block** dont le *containing block* est plus large, il faut mettre les marges horizontales en **auto** : elles se règlent alors automatiquement pour compléter la largeur manquante entre le **block** courant et le *containing block*. Ceci n'est pas valable pour les marges verticales.

5. Rajoutez des règles CSS pour que les fils **<div>** enfants de **<nav>** aient une couleur de fond **#5BBDBF**,
6. Ajoutez une règle CSS pour que les éléments **<a>** descendants de **<nav>** aient la couleur de fond **#c0d5c2**.

- **fixed** : le reste de la page fait comme si l'élément n'existait pas. L'élément se positionne relativement à la fenêtre d'affichage ; il paraît donc *fixé* lors d'un défilement de la page.

Un élément est dit **positionné** s'il a une position autre que **static** (qui est la valeur par défaut). Pour indiquer le décalage de position, on utilise les propriétés **top**, **left**, **right** et **bottom**. Par exemple, les propriétés

```
position:relative;  
top:20px;  
left:20px;
```

vont positionner un élément **20px** plus à droite et en bas qu'il n'aurait dû l'être.

Référence : [Mozilla Developer Network \(MDN\)](#)

Exercice 6

1. Donnez aux sous-menus la couleur de fond **#aca**. Puis masquez-les par défaut en CSS.
2. Réaffichez le premier sous-menu (resp. le deuxième) avec **display:block** lorsque la souris passe au-dessus de “accueil” (resp. “contact”) en CSS.

Aide : En pratique, nous allons vérifier si le père **<div>** du sous-menu est survolé (**:hover**). Nous souhaitons donc un sélecteur CSS qui sélectionne les éléments de classe **submenu** qui sont fils d'un **<div>** survolé qui sont fils d'un **<nav>**.

À ce stade les sous-menus apparaissent bien lorsque l'on survole les éléments “Accueil” et “Contact”. Par contre il n'est pas possible d'entrer dans ces sous-menus. Nous consacrons toute la prochaine section au comportement **display:flex** qui va permettre d'améliorer l'apparence de notre menu et de remédier à ce problème d'accessibilité.

4.display:flex

La valeur de display **flex** correspond au layout appelé FlexBox. Appliquée à un élément, la valeur de display **flex** va permettre de modifier la disposition **de ses enfants**. C'est donc une différence fondamentale avec les valeurs **block** et **inline**, qui eux avaient un impact directement **sur l'élément lui-même**.

Lorsque le display est **flex**, on peut par exemple :

- fixer le sens de rendu des enfants,
- l'espace entre ces derniers,
- leurs centrages dans les différentes directions,
- modifier leurs ordres d'apparition, ...

Nous précisons par la suite quelques-unes de ces contraintes/propriétés.

4.1La propriété flex-direction

La propriété **flex-direction** permet de préciser si les enfants vont se mettre en ligne ou en colonne et dans quel ordre. Ses valeurs sont :

- **row** : mise en ligne dans l'ordre classique de gauche à droite ;
- **row-reverse** : mise en ligne dans l'ordre inversé ;
- **column** : mise en colonne dans l'ordre classique de haut en bas ;
- **column-reverse** : mise en colonne dans l'ordre inversé.

L'attribut **flex-direction** s'appliquant aux enfants, il rentre en conflit avec les valeurs de display **block** ou **inline** de ces derniers. L'exercice suivant va nous permettre entre autres de savoir qui surcharge l'autre en cas de conflit.

Exercice 8

Dans cet exercice, nous souhaitons que les `<div>` titres des menus soient centrés verticalement mais qu'ils prennent toute la hauteur.

1. Pour mieux voir le comportement, nous allons donner jusqu'à la fin de cet exercice une hauteur de **200px** et la couleur de fond **#FF00FF** à `<nav>` et, pour les sous-menus, un décalage de **200px** vers le bas.
2. Centrez les éléments enfants de `<nav>` à l'aide de la propriété **align-items** de `<nav>`. Que constatez-vous sur la hauteur des `<div>` et l'accessibilité des sous-menus avec la souris ? Est-ce que le `<div>` est centré verticalement ?

Note : Si rien de ne se passe, utiliser l'inspecteur du navigateur pour comprendre ce qui est centré (la marge automatique placée sur le `<nav>` par exemple peut expliquer des choses).

3. Utilisez maintenant la valeur **stretch**. Que constatez-vous sur la hauteur des `<div>` et l'accessibilité des sous-menus avec la souris ? Est-ce que le `<div>` est centré verticalement ?
4. Faites en sorte que les sous-menus restent accessibles et que les textes "Accueil" et "Contact" soient centrés verticalement dans la barre de navigation. Pour ceci :
 - i. Faites en sorte que les `<div>` prennent toute la hauteur à l'aide des 2 question précédentes.
 - ii. Centrez verticalement les liens `<a>` au sein des `<div>` : il faut pour cela que les `<div>` enfants du `<nav>` soient **flex**, ce qui va permettre de centrer verticalement ses enfants `<a>` à l'aide de l'un des 2 comportements vues précédemment.
5. Annulez (commentez) les propriétés temporaires de la question 1.

4.3 La propriété justify-content

Cette propriété permet de préciser la façon dont les enfants vont se disposer dans la direction donnée par **flex-direction**.

Les valeurs possibles sont :

- **flex-start** : au début de l'axe donné par **flex-direction** ;
- **flex-end** : à la fin de cet axe ;
- **center** : centré sur cet axe ;
- **space-between** : les éléments vont du début à la fin de l'axe en rajoutant des espaces entre les éléments ;
- **space-around** : les éléments vont du début à la fin de l'axe en rajoutant des espaces autour des éléments (donc entre les éléments et avant le premier et après le dernier) ;

Exercice 11

1. Donnez au body la **width** de **900px**.
2. Déplacez dans le HTML la section contenant la **<table>** dans **<aside>** si cela n'est pas déjà fait.
3. Utilisez la valeur de **display flex** sur la balise **<main>** pour que ses enfants **<article>** et **<aside>** s'affichent comme deux colonnes côte à côte.
4. Fixez la largeur de **<article>** à **60%**, et celle de **<aside>** à **30%**. Ce dernier élément aura une marge gauche de **10%**.
5. Donnez à **<aside>** et à **<article>** la couleur de fond **#CCC**.

6.Cacher ou enlever un élément du rendu

Il existe plusieurs façons de faire disparaître de l'écran un élément HTML avec un bloc de déclaration :

- **display:none**
- **visibility:hidden**

Nous avons déjà utilisé le bloc de déclaration **display:none** dans le menu pour cacher un sous-menu sans que ce dernier marque la page par son absence. Le bloc de déclaration **visibility:hidden** quant à lui laisse l'espace innoccupé à la place de l'élément.

Voyons un usage de **visibility:hidden** :

Exercice 12

Nous voulons marquer visuellement le menu sous la souris par une petite puce dans les **<a>** du menu.

```
<span class="puce">■</span>
```

Elle se positionne à gauche du texte, elle n'est visible que lorsque la souris survole le **<div>** contenant. Dans le cas contraire l'espace reste occupé (pour ne pas faire un effet de flicker/tremblement au survol du menu)

1. Ajoutez cette puce devant le texte des liens **<a>**,
2. Ajoutez **visibility:hidden** à l'élément de classe **puce** et la valeur **visible** sur le survole de la souris sur le **<div>** parent.
3. Supprimez la couleur de fond sur les balises **<a>** pour plus de lisibilité.
4. (Optionnel) Styliser les sous-menus.

7.Fini !

Le mot de la fin à propos de flex ?

« Moi je stoppe sur mon flex. Quand je sticks ma vibes le dancefloor se breaks et se fluxe dans la vibes. » *Gad Elmaleh*

1. En fait le HTML5 permet cette inclusion dans [certains cas](#). 

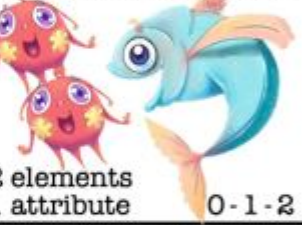
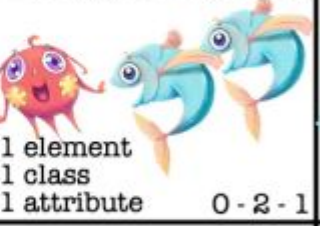
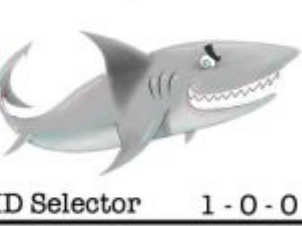
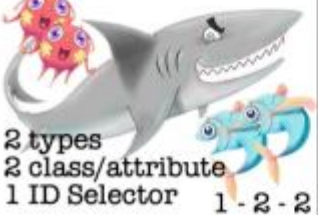
Aide :

Outil en ligne qui calcule automatiquement le score d'une règle CSS, ou pour utiliser le terme exact, le poids d'un sélecteur :

<https://specificity.keegan.st>

CSS SPECIFISHITY

WITH PLANKTON, FISH AND SHARKS

*  universal selector 0 - 0 - 0	div  1 element 0 - 0 - 1	li > ul  2 elements 0 - 0 - 2	body div ... ul li p a  12 elements 0 - 0 - 12
.myClass  1 class 0 - 1 - 0	*.myClass  1 universal selector 1 class 0 - 1 - 0	[type=checkbox]  1 attribute selector 0 - 1 - 0	:only-of-type  1 pseudo-class 0 - 1 - 0
li.myClass  1 element 1 class 0 - 1 - 1	li[attr]  1 element 1 attribute 0 - 1 - 1	li:nth-of-type(3n)~li  2 elements 1 pseudo-class 0 - 1 - 2	form input[type=email]  2 elements 1 attribute 0 - 1 - 2
li.class:nth-of-type(3n)  1 element 1 class 1 pseudo-class 0 - 2 - 1	input[type]:not(.class)  1 element 1 class 1 attribute 0 - 2 - 1	cl:nth-child(4n)chk[type]...  10 class/attribute/pseudo-classes 0 - 10 - 0	#myDiv  ID Selector 1 - 0 - 0
#myDiv li.class a[href]  2 types 2 class/attribute 1 ID Selector 1 - 2 - 2	#divitis #myDiv a  2 ID Selectors 1 type selector 2 - 0 - 1	style=""  inline style 1 - 0 - 0 - 0	!important  important 1 - 0 - 0 - 0 - 0

ESTELLE WEYL * @ESTELLEWV * WWW.STANDARDISTA.COM * 2104

X-0-0: The number of ID selectors

0-Y-0: The number of class selectors, attributes selectors, and pseudo-classes

0-0-Z: The number of type selectors and pseudo-elements

*, +, >, ~ : Universal selector and combinators do not increase specificity

:not(x): Negation selector has no value. Argument increases specificity



Révision #2

Créé Tue, Sep 23, 2025 2:52 PM par [Rodolphe](#)

Mis à jour Wed, Oct 1, 2025 9:07 AM par [Rodolphe](#)